AI ASSISTED CODING

ASSIGNMENT – 19.1

NAME: Varshitha. G

HTNO: 2303A51103

Task 1 – Python to Java Conversion

Task: Translate a simple Python class into Java using AI assistance.

Instructions:

• Prompt AI to convert a Python class with attributes and methods into equivalent Java code.

• Verify that constructors, method signatures, and data types are properly handled.

• Run both versions (Python & Java) to compare expected outputs.

Expected Output:

• Equivalent working Java class translated from Python.

```python
class BookCollection:
    def __init__(self):
        self.books = []

    def add_book(self, book: str) -> None:
        self.books.append(book)

    def remove_book(self, book: str) -> None:
        if book in self.books:
            self.books.remove(book)
        else:
            print(f"Book '{book}' not found in the collection.")

    def list_books(self) -> list:
        return self.books
# Example usage:
if __name__ == "__main__":
    bc = BookCollection()
    bc.add_book("To Kill a Mockingbird")
    bc.add_book("1984")
    print(bc.list_books())  # Output: ['To Kill a Mockingbird', '1984']
    bc.remove_book("1984")
    print(bc.list_books())  # Output: ['To Kill a Mockingbird']
    bc.remove_book("The Great Gatsby")  # Output: Book 'The Great Gatsby' not found in the collection.
```

```java
public class BookCollection {
    private List<String> books;

    public BookCollection() {
        this.books = new ArrayList<>();
    }

    public void addBook(String book) {
        books.add(book);
    }

    public void removeBook(String book) {
        if (books.contains(book)) {
            books.remove(book);
        } else {
            System.out.println("Book '" + book + "' not found in the collection.");
        }
    }

    public List<String> listBooks() {
        return books;
    }

    public static void main(String[] args) {
        BookCollection bc = new BookCollection();
        bc.addBook("To Kill a Mockingbird");
        bc.addBook("1984");
        System.out.println(bc.listBooks()); // Output: [To Kill a Mockingbird, 1984]

        bc.removeBook("1984");
        System.out.println(bc.listBooks()); // Output: [To Kill a Mockingbird]

        bc.removeBook("The Great Gatsby"); // Output: Book 'The Great Gatsby' not found in the collection.
    }
```

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:
['To Kill a Mockingbird', '1984']
['To Kill a Mockingbird']
Book 'The Great Gatsby' not found in the collection.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

• Ask AI to translate a recursive C++ factorial function into Python.

• Validate correctness by testing with sample inputs.

• Ensure Python version follows Pythonic conventions.

Expected Output:• A correct Python implementation equivalent to the original C++ factorial function.

```cpp
#include <iostream>
using namespace std;
int factorial(int n) {
    if (n == 0 || n == 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
int main() {
    int number;
    cout << "Enter a number to calculate its factorial: ";
    cin >> number;
    cout << "Factorial of " << number << " is " << factorial(number) << endl;
    return 0;
}
```

```python
def factorial(n: int) -> int:
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
if __name__ == "__main__":
    number = int(input("Enter a number to calculate its factorial: "))
    print(f"Factorial of {number} is {factorial(number)}")
```

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/var
Enter a number to calculate its factorial: 5
Factorial of 5 is 120
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/var
Enter a number to calculate its factorial: 7
Factorial of 7 is 5040
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

• Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).

• Ask AI to generate equivalent Pandas code.

• Test the generated code on a sample DataFrame.

Expected Output:

• Equivalent Pandas query that retrieves the correct subset of data.

```sql
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    salary DECIMAL(10, 2) NOT NULL
);
INSERT INTO employees (name, salary) VALUES ('Alice', 60000);
INSERT INTO employees (name, salary) VALUES ('Bob', 45000);
INSERT INTO employees (name, salary) VALUES ('Charlie', 55000);
SELECT name, salary FROM employees WHERE salary > 50000;
```

name	salary
Alice	60000
Charlie	55000

```python
import pandas as pd
data = {
    'employee_id': [1, 2, 3],
    'name': ['Alice', 'Bob', 'Charlie'],
    'salary': [60000, 45000, 55000]
}
df = pd.DataFrame(data)
result = df[df['salary'] > 50000][['name', 'salary']]
print(result)
```

```
PS C:\Users\varsh\OneD
      name  salary
0    Alice   60000
2  Charlie   55000
```

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will choose a real-time use case where they need code in

multiple languages. Example:• Converting a Python web scraping snippet into JavaScript for browser automation.

• Translating a Java method into C# for enterprise applications.

Instructions:

• Prompt AI to translate code between two chosen languages.

• Test both implementations to ensure they produce the same results.

• Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

• Working equivalent code snippets in at least two different programming languages.

```python
def print_natural_numbers(n: int) -> None:
    """
    Prints the first N natural numbers.

    Parameters:
    n (int): The number of natural numbers to print
    """
    for i in range(1, n + 1):
        print(i)
# Example usage:
if __name__ == "__main__":
    N = 10
    print_natural_numbers(N)  # Output: 1 2 3 4 5 6 7 8 9 10
```

```c
#include <stdio.h>
void print_natural_numbers(int n) {
    /*
    Prints the first N natural numbers.

    Parameters:
    n (int): The number of natural numbers to print
    */
    for (int i = 1; i <= n; i++) {
        printf("%d ", i);
    }
    printf("\n");
}
int main() {
    int N = 10;
    print_natural_numbers(N);  // Output: 1 2 3 4 5 6 7 8 9 10
    return 0;
}
```

```go
import "fmt"
func printNaturalNumbers(n int) {
    /*
    Prints the first N natural numbers.

    Parameters:
    n (int): The number of natural numbers to print
    */
    for i := 1; i <= n; i++ {
        fmt.Printf("%d ", i)
    }
    fmt.Println()
}
func main() {
    N := 10
    printNaturalNumbers(N) // Output: 1 2 3 4 5 6 7 8 9 10
}
```

```java
public class Main {
    public static void printNaturalNumbers(int n) {
        /*
        Prints the first N natural numbers.

        Parameters:
        n (int): The number of natural numbers to print
        */
        for (int i = 1; i <= n; i++) {
            System.out.print(i + " ");
        }
        System.out.println();
    }
    public static void main(String[] args) {
        int N = 10;
        printNaturalNumbers(N); // Output: 1 2 3 4 5 6 7 8 9 10
    }
}
```

```
1
2
3
4
5
6
7
8
9
10
```

In this task, we selected a real-time use case and converted code from one programming language to another using AI. We ensured that both versions of the code perform the same functionality by testing them. During the process, we observed differences in syntax, libraries, and structure between the languages. Some challenges included handling language-specific functions and adapting logic accordingly. Overall, the translation helped us understand cross-language development and problem-solving.